Arquitectura y API · arce.dev

Oct 27, 2026 · @Ezequiel

Documento de arquitectura del Trabajo Final Integrador de Lenguajes IV (UCASAL, 2026): un portfolio fullstack con sitio público, área de usuarios registrados y paneles de operador y administrador.

1. Qué es el sistema y cómo está armado

El sistema es el sitio profesional de Martina Arce, desarrolladora fullstack, armado en tres piezas independientes que se hablan por HTTP y JSON: un **cliente** en React, una **API REST** en Node y una **base de datos** MongoDB. Cualquier visitante ve su perfil, su stack, sus servicios y un listado de proyectos; quien se registra ve el detalle y las demos; cualquiera puede escribirle por un formulario; un **operador** responde esas consultas por email; y un **administrador** gestiona el contenido, las cuentas y un dashboard con contadores y gráficos.

Estado al 3/11/2026. La API del servidor está completa, mergeada en `main` y publicada en Render (`https://portfolio-fullstack-cidz.onrender.com`), con la base en Atlas y el cliente publicado en Vercel (`https://portfolio-fullstack-kappa-wine.vercel.app`). 271 tests automáticos, una revisión de seguridad hecha y el smoke contra lo publicado en verde. Cloudinary está configurado; faltan probar con credenciales reales los emails (Brevo) y el login con Google. El cliente lo desarrollan A, B y D (sección 11).

Cambios al adaptar la API al frontend (3/11)

- **Formato de error:** `{ ok: false, error: "mensaje", code, details: [{ field, message }] }`, con `field` igual al nombre del campo del formulario. `error` es un texto.
- **Objetos y listas:** todo objeto trae `_id` (e `id`); usuarios y consultas se listan como `data: { items, page, limit, total, pages }`.
- **Formularios en español:** Contacto (`nombre`, `telefono`, `tipo`, `presupuesto`, `asunto`, `mensaje`, adjunto en `adjunto`) y Registro (`nombre`). Los errores vuelven con esos nombres.
- **Multipart:** proyectos (`images`, `keepImages`) y perfil (`photo`) se suben a Cloudinary desde el propio formulario; también se acepta JSON.
- **Servicios:** `icon` (code, server, database, chat), `color` (carmin, negro, vicuna, exito, oro) y `order` que guarda el panel al reordenar.

- **Contraseña:** cambiarla exige la actual si la cuenta ya tiene una (quien entró solo con Google crea la primera sin ella) y cierra las otras sesiones; la respuesta trae un `token` nuevo.
- **Google:** vincularlo no borra la contraseña. **Eliminar una cuenta** anonimiza sus consultas en vez de borrarlas, así siguen en la bandeja y en las estadísticas.
- El detalle de cada endpoint está en la sección 0 del documento *Contrato de API*, que manda sobre este texto si hay diferencias.

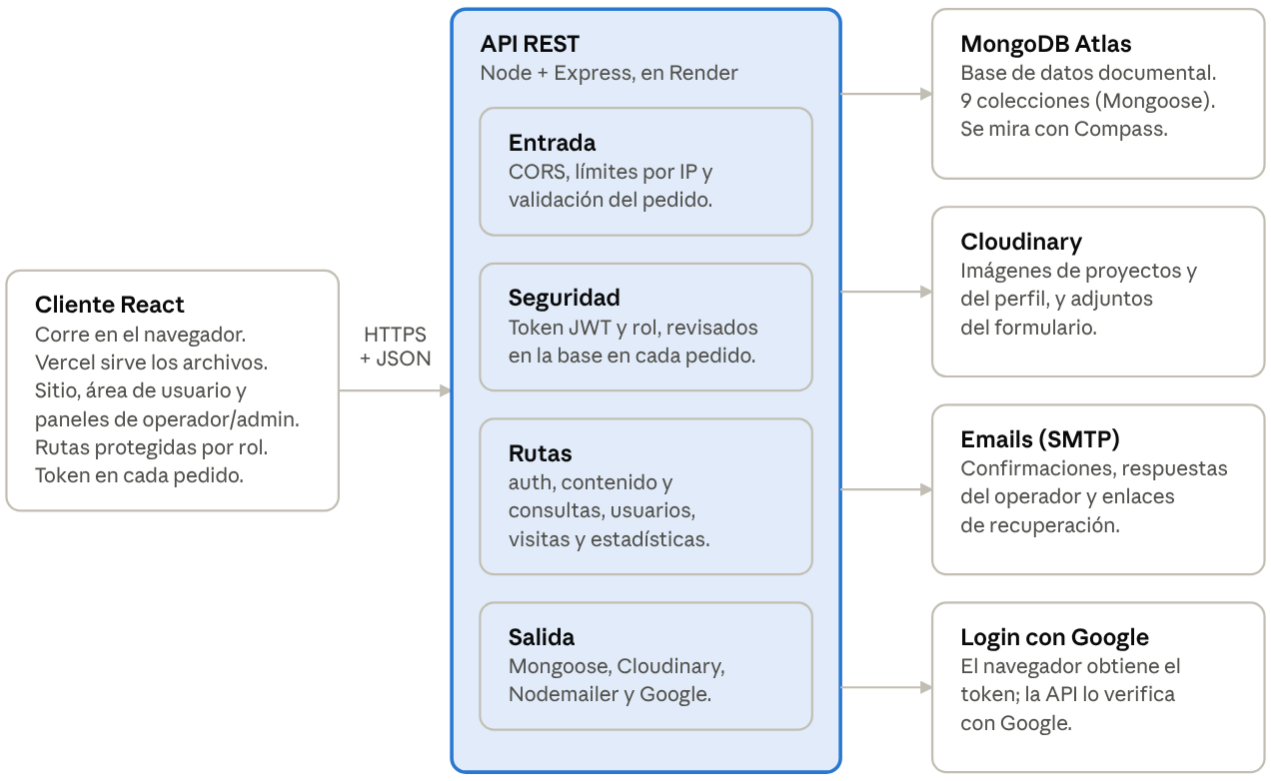
Qué cubre de la consigna

Requisito	Cómo se resuelve	Dónde
Registro con base propia y con Google	Cuentas con email y contraseña (bcrypt) o con Google Identity Services, y sesión con JWT.	API `/api/auth` y páginas de login y registro
Proyectos y demos solo para registrados	El listado público es un resumen; el detalle y la demo exigen sesión.	API y ruta protegida `/proyectos/:id`
Formulario de contacto por email	La consulta se guarda y se manda un email de confirmación a quien escribió.	API `/api/inquiries` y página de contacto
Operador: gestión y respuestas por email	Bandeja con filtros y estados; la respuesta sale por email y queda en el historial.	API y panel `/operador`
CRUD de proyectos, demos y servicios	El admin crea, edita, oculta y borra; las imágenes se guardan en Cloudinary.	API y paneles `/admin/*`
Gestión de usuarios	Listar, cambiar rol, activar o desactivar y eliminar cuentas.	API `/api/users` y `/admin/usuarios`
Contador de visitas y de accesos autenticados	Una visita por sesión y página; cada inicio de sesión queda registrado.	`/api/visits` y colección `logins`

Requisito	Cómo se resuelve	Dónde
Consultas parametrizadas y gráficos	Estadísticas por rango de fechas y tipo: visitas por día o semana, consultas por tipo y registros por semana.	API `/api/stats` y dashboard con Chart.js
Validación estricta de campos, archivos e imágenes	Se valida en el cliente y siempre en la API; los archivos, por su contenido real.	API (sección 7)
Animaciones fluidas y APIs Web	anime.js, Web Animations API, Intersection Observer y Lottie; Fetch, Clipboard y Web Share.	Cliente
Base de datos y despliegue en la nube	MongoDB Atlas (documental); Vercel y Render.	Secciones 5 y 8
Manual de marca y prototipo	PDF y prototipo navegable de 27 pantallas.	Entregable de A

2. Arquitectura

El navegador solo habla con la API. La API revisa cada pedido (CORS, límites, token y rol) y es la única pieza que usa la base, Cloudinary, el servicio de emails y Google. El cliente se publica en Vercel, la API en Render y la base vive en Atlas.



arquitectura · cliente, API y cuatro servicios

Un pedido de punta a punta

1. El cliente llama a la API con `fetch` y manda el token de sesión en el encabezado `Authorization`.
2. La API comprueba que el origen esté permitido (CORS), que no se pasó de los límites de pedidos y que el JSON sea válido.
3. Verifica el token y busca a la persona en la base: si su cuenta se desactivó, el pedido se rechaza aunque el token sea válido. Después comprueba que su rol alcance para ese endpoint.
4. Valida los campos y hace el trabajo: leer o guardar en MongoDB, subir un archivo a Cloudinary o mandar un email.
5. Responde siempre `{ ok, data }` o `{ ok: false, error }`, con un código estable y un mensaje listo para mostrar.

3. Tecnologías

El servidor usa Node.js con Express y MongoDB con Mongoose; el cliente usa React con Vite y React Router; y todo se publica en servicios gratuitos. Las versiones del servidor son las del `package.json`.

Servidor y servicios

Capa	Tecnología	Para qué se usa
Plataforma	Node.js 20.6 o más (probado con 24) y Express 5	API REST, rutas y middlewares.
Base de datos	MongoDB Atlas y Mongoose 9	Persistencia documental, esquemas con validaciones y índices. Compass para mirar los datos.
Autenticación	`bcryptjs`, `jsonwebtoken` (HS256) y `google-auth-library`	Contraseñas con hash, sesión con token y verificación del token de Google.
Seguridad HTTP	`helmet`, `cors` y `express-rate-limit`	Encabezados de seguridad, orígenes permitidos y límites de pedidos.
Archivos	`multer` y `cloudinary`	Lectura del archivo en memoria, validación por su contenido real y almacenamiento.
Emails	Nodemailer	Envío por SMTP: Gmail en desarrollo, un servicio con el puerto 2525 en Render.
Validación	Funciones propias	Las reglas y los mensajes del contrato de API, que son los del prototipo.
Pruebas	`node:test`, `supertest` y `mongodb-memory-server`	246 tests que corren sin internet ni base instalada.

Cliente (a cargo de A, B y D)

Según el plan, el manual de marca y el prototipo. El código del cliente no está descrito acá.

Capa	Tecnología	Para qué se usa
Interfaz	React y Vite	Componentes funcionales con `useState` y `useEffect`; proyecto creado con `npm create vite@latest`.

Capa	Tecnología	Para qué se usa
Navegación	React Router	Layouts para el sitio, el operador y el admin; rutas protegidas por rol; página 404.
Estilos	Framework CSS (Bootstrap 5)	La consigna lo exige. El plan dice Tailwind y el manual dice Bootstrap 5: falta cerrar cuál.
Animaciones	anime.js, Web Animations API, Intersection Observer, Lottie y transiciones CSS	Entrada del título, aparición al hacer scroll, tarjetas, contadores y confirmación de envío.
Datos	Fetch API	Todos los pedidos a la API.
Gráficos	Chart.js	Visitas, consultas por tipo y registros en el dashboard.
Tipografía	Syne, Instrument Sans y JetBrains Mono	Del manual de marca.

Publicación

Pieza	Servicio	Nota
Cliente	Vercel	Despliegue automático con cada push a `main`.
API	Render (plan gratuito)	Se duerme tras 15 minutos sin tráfico y tarda cerca de 1 minuto en despertar.
Base de datos	MongoDB Atlas (M0)	Gratuita.
Archivos	Cloudinary	Gratuito.
Código	GitHub	Ramas por tarea, Pull Requests revisados por otro integrante y `main` protegida.

Teoría de las clases y dónde se aplica

Clase	Tema	Dónde se aplica
1	API Web y modelo REST	Arquitectura cliente-servidor: el cliente consume la API propia.

Clase	Tema	Dónde se aplica
2	REST, JSON, CRUD, códigos de error y parámetros de consulta	API en Express con CRUD, respuestas JSON `{ ok, data }`, códigos HTTP (400, 401, 403, 404, 409, 413, 415, 429, 500) y filtros como `?tipo=cotizacion&estado=nueva`.
3	Web APIs, Fetch, Web Animations y anime.js	`fetch` para la API y las animaciones del sitio.
4 y 5	React, componentes, props, state y hooks	Todo el cliente; tarjetas y grillas reutilizables alimentadas desde la base.
6 y 7	React Router, layouts, rutas anidadas, 404 y `useParams`	Navegación del sitio, `/proyectos/:id`, paneles anidados y rutas protegidas.
8	Vite	Estructura del cliente.
9	MongoDB: documentos, colecciones, objetos anidados y Compass	Base en Atlas; las respuestas del operador van anidadas dentro de cada consulta.

4. Roles y permisos

Hay cuatro roles y quien decide es la API: el cliente solo oculta lo que no corresponde, pero cada endpoint vuelve a comprobar la sesión y el rol. El rol se lee de la base en cada pedido, no del token, así que un cambio de rol o una cuenta desactivada valen de inmediato.

Qué puede hacer	Visitante	Usuario	Operador	Admin
Ver perfil, stack, servicios, experiencia y el listado de proyectos (resumen)	Sí	Sí	Sí	Sí
Enviar una consulta (con sesión, queda en su cuenta)	Sí	Sí	Sí	Sí
Ver el detalle y la demo de un proyecto	No	Sí	Sí	Sí
Ver sus consultas y las respuestas, cambiar su nombre y su contraseña	No	Sí	Sí	Sí
Eliminar su propia cuenta	No	Sí	No	No
Ver la bandeja, filtrar, responder consultas y cambiar su estado	No	No	Sí	Sí

Qué puede hacer	Visitante	Usuario	Operador	Admin
Subir imágenes de proyectos y del perfil	No	No	No	Sí
Crear, editar, ocultar y borrar proyectos y servicios; editar el perfil público	No	No	No	Sí
Gestionar usuarios: cambiar rol, activar o desactivar, eliminar	No	No	No	Sí
Ver el dashboard y las estadísticas	No	No	No	Sí

Reglas que cuidan los permisos:

- Una cuenta desactivada no puede iniciar sesión y su sesión abierta deja de valer en el próximo pedido.
- Un admin no puede quitarse el rol, desactivarse ni eliminarse a sí mismo, así que siempre queda un administrador activo.
- El operador y el admin no pueden eliminar su propia cuenta desde «Mi cuenta»: la elimina un admin desde Usuarios.
- Un proyecto en borrador es un 404 para cualquiera que no sea admin, para no revelar que existe.
- El detalle de un proyecto (demo, repositorio y textos largos) no sale en el listado público: solo lo devuelve la API a una sesión iniciada.

Rutas del cliente

Según el manual de marca y el prototipo. Las rutas protegidas mandan al login y, después de entrar, vuelven a la página pedida; cada rol ve solo su panel, salvo el admin, que también entra a `/operador`.

Ruta	Página	Acceso
`/`	Inicio: perfil, stack, servicios y experiencia	Público
`/proyectos`	Listado de proyectos (resumen)	Público
`/proyectos/:id`	Detalle del proyecto y demo	Registrado
`/servicios`	Servicios	Público
`/contacto`	Formulario de contacto	Público

Ruta	Página	Acceso
`/login`, `/registro`, `/recuperar`	Ingreso (email o Google), alta de cuenta y recuperación de contraseña	Público
`/cuenta`	Mi cuenta: datos, seguridad, mis consultas y eliminar la cuenta	Registrado
`/operador` y `/operador/consultas/:id`	Bandeja de consultas y detalle con respuesta por email	Operador y admin
`/admin`	Dashboard con contadores y gráficos	Admin
`/admin/proyectos` y `/admin/servicios`	Altas, ediciones y bajas	Admin
`/admin/usuarios`	Gestión de usuarios	Admin
`/admin/perfil`	Edición del perfil público	Admin
`*`	Página 404	Público

5. Modelo de datos

La base es documental (MongoDB) y tiene nueve colecciones. Las cuentas son el centro: los accesos, las visitas y las consultas apuntan a `users` con un `userId`, y las respuestas del operador no son una colección aparte sino una lista dentro de cada consulta, porque siempre se leen junto a ella.



modelo de datos · 9 colecciones y sus relaciones

Colecciones y campos principales

Colección	Qué guarda	Campos principales	Reglas que importan
`users`	Las cuentas de los cuatro roles	`name`, `email` (único, en minúsculas), `passwordHash`, `googleId`, `role` (`user`, `operator`, `admin`), `active`, `newsletter`, `loginCount`, `lastLogin`, `tokenVersion`	El hash nunca sale en una respuesta. Una cuenta puede tener contraseña, Google o las dos. `tokenVersion` cierra las sesiones al cambiar la contraseña.
`logins`	Cada inicio de sesión correcto	`userId`, `method` (`password` o `google`), `at`	Permite contar los «accesos autenticados» de un período.

Colección	Qué guarda	Campos principales	Reglas que importan
`projects`	Proyectos y demos	`title`, `summary`, `problem`, `solution`, `highlights`, `role`, `duration`, `stack`, `images` (hasta 5, con `url` y `publicId`), `demoUrl`, `repoUrl`, `visible`, `featured`	`visible: false` es un borrador. Las URLs son `https`.
`services`	Servicios que ofrece	`name`, `description`, `color`, `priceFrom`, `order`, `visible`	El nombre es único sin distinguir mayúsculas ni tildes. `priceFrom` vacío significa «a consultar».
`profile`	El perfil público (un solo documento)	`name`, `title`, `tagline`, `bio`, `email`, `location`, `github`, `linkedin`, `photo`, `stack`, `experience`	Si el admin todavía no guardó nada, la API devuelve el del seed.
`inquiries`	Consultas del formulario	`number`, `name`, `email`, `phone`, `type` (`general`, `cotizacion`, `asesoria`), `budget`, `subject`, `message`, `attachment`, `status` (`nueva`, `en_proceso`, `respondida`, `cerrada`), `userId`, `replies`	`number` es correlativo (el «N.º 0031» de los emails). El adjunto tiene que ser un archivo subido por nosotros.
`visits`	Visitas al sitio	`path`, `userId`, `sessionId`, `createdAt`	Un índice único sobre `sessionId` y `path`: recargar la misma página no suma.
`counters`	Numeración de las consultas	`_id`, `seq`	Se incrementa de forma atómica.
`loginattempts`	Intentos de login fallidos	`email`, `ip`, `at`	Un índice TTL los borra a los 15 minutos.

6. API REST

La API tiene 40 endpoints bajo `/api`, todos en JSON (menos la subida de archivos, que es `multipart/form-data`). El detalle campo por campo, con cuerpos de ejemplo y mensajes de error, está en el documento Contrato de API · arce.dev; acá está el mapa completo.

Convenciones

- **Sesión:** `Authorization: Bearer <token>`. El token (JWT) se obtiene al registrarse o iniciar sesión y dura 7 días.
- **Respuesta correcta:** `{ "ok": true, "data": ... }`. Todo objeto trae `_id` e `id`. Las listas de usuarios y consultas devuelven `data: { items, page, limit, total, pages }`; la de proyectos, un array con `"meta": { "page", "limit", "total", "totalPages" }`. Se piden con `?page=1&limit=20` (máximo 100).
- **Error:** `{ "ok": false, "error": "mensaje", "code": "validacion", "details": [{ "field", "message" }] }`. El `error` es un texto en español listo para mostrar, el `code` es estable para el programa y `details` trae un mensaje por campo cuando la validación falla, con el nombre del campo del formulario, para pintarlo junto al input.
- **Fechas:** ISO 8601 en UTC. Los días de las estadísticas se cuentan en hora de Salta (UTC−3).

HTTP	`error.code`	Cuándo
400	`validacion`, `token_invalido`, `pedido_invalido`	Un campo no cumple las reglas, el enlace de recuperación venció, o el pedido no se pudo leer.
401	`no_autenticado`, `credenciales_invalidas`	Falta la sesión o venció; email o contraseña incorrectos.
403	`sin_permiso`, `cuenta_desactivada`	El rol no alcanza, o la cuenta fue desactivada.
404	`no_encontrado`	No existe, o no es visible para quien pregunta.
409	`email_en_uso`, `nombre_en_uso`	Ya existe una cuenta con ese email o un servicio con ese nombre.
413	`archivo_pesado`, `pedido_muy_grande`	Un archivo de más de 2 MB, o un pedido JSON de más de 100 KB.
415	`formato_no_permitido`	El archivo no es de un tipo permitido.

HTTP	`error.code`	Cuándo
429	`demasiados_intentos`	Se pasó de un límite; trae `retryAfterSeconds`.
502 y 503	`email_no_enviado`, `almacenamiento_error`, `almacenamiento_no_configurado`, `google_no_configurado`	Falla un servicio externo o falta configurarlo.
500	`error_interno`	Falla inesperada; el mensaje nunca trae detalles técnicos.

Autenticación y cuenta

Método	Endpoint	Acceso	Qué hace
POST	`/api/auth/register`	Público	Crea la cuenta y abre sesión.
POST	`/api/auth/login`	Público	Abre sesión con email y contraseña; bloquea tras 5 fallos desde una misma conexión.
POST	`/api/auth/google`	Público	Abre sesión (o crea la cuenta) con el token de Google.
GET	`/api/auth/me`	Registrado	Devuelve la cuenta de la sesión.
PATCH	`/api/auth/me`	Registrado	Cambia el nombre y la preferencia de newsletter.
POST	`/api/auth/password`	Registrado	Crea o cambia la contraseña; cierra las demás sesiones y devuelve un token nuevo.
DELETE	`/api/auth/me`	Usuario	Elimina la cuenta; sus consultas quedan anonimizadas.
POST	`/api/auth/forgot`	Público	Envía por email un enlace de recuperación que vence en 1 hora.
POST	`/api/auth/reset`	Público	Fija una contraseña nueva con el token del enlace.

Contenido: perfil, proyectos y servicios

Método	Endpoint	Acceso	Qué hace
GET	`/api/profile`	Público	Perfil del portfolio para la home.
PUT	`/api/profile`	Admin	Guarda el perfil completo.
GET	`/api/projects`	Público	Proyectos visibles en resumen: destacados primero; filtro `?stack=`; paginado.
GET	`/api/projects/:id`	Registrado	Detalle completo con demo y repositorio.
POST	`/api/projects`	Admin	Crea un proyecto (queda como borrador si no se pide visible).
PUT	`/api/projects/:id`	Admin	Modifica los campos que se envían.
DELETE	`/api/projects/:id`	Admin	Elimina el proyecto y sus imágenes de Cloudinary.
GET	`/api/services`	Público	Servicios visibles, en orden.
POST	`/api/services`	Admin	Crea un servicio al final de la lista.
PUT	`/api/services/:id`	Admin	Modifica los campos que se envían.
PATCH	`/api/services/order`	Admin	Reordena con la lista completa de ids.
DELETE	`/api/services/:id`	Admin	Elimina el servicio y renumera los demás.
GET	`/api/admin/projects`	Admin	Todos los proyectos, borradores incluidos; filtros `?q=` y `?visible=`.
GET	`/api/admin/services`	Admin	Todos los servicios, ocultos incluidos.

Consultas y archivos

Método	Endpoint	Acceso	Qué hace
POST	`/api/inquiries`	Público	Guarda la consulta y manda la confirmación por email. Con sesión, queda en la cuenta.
GET	`/api/inquiries`	Operador	Bandeja con filtros `tipo`, `estado`, `desde`, `hasta`, `q` y paginación.

Método	Endpoint	Acceso	Qué hace
GET	`/api/inquiries/mine`	Registrado	Las consultas de la sesión, con la última respuesta.
GET	`/api/inquiries/:id`	Operador	Detalle con el historial de respuestas.
POST	`/api/inquiries/:id/reply`	Operador	Responde por email y guarda la respuesta; pasa a «respondida».
PATCH	`/api/inquiries/:id`	Operador	Cambia el estado.
POST	`/api/uploads`	Público y Admin	Sube un archivo. Imágenes de proyecto y perfil: solo admin. Adjuntos de contacto: cualquiera.

Usuarios, visitas y estadísticas

Método	Endpoint	Acceso	Qué hace
GET	`/api/users`	Admin	Lista con filtros `rol` y `q`, y paginación.
GET	`/api/users/:id`	Admin	Una cuenta.
PATCH	`/api/users/:id`	Admin	Cambia el rol o activa y desactiva la cuenta.
DELETE	`/api/users/:id`	Admin	Elimina la cuenta; sus consultas quedan anonimizadas.
POST	`/api/visits`	Público	Registra una visita: una por sesión y página.
GET	`/api/stats/summary`	Admin	Las cuatro tarjetas: visitas, usuarios, accesos y consultas del período.
GET	`/api/stats/visits`	Admin	Serie de visitas por día o por semana.
GET	`/api/stats/inquiries-by-type`	Admin	Consultas por tipo.
GET	`/api/stats/registrations`	Admin	Cuentas creadas por semana.
GET	`/api/health`	Público	Dice que el servidor está arriba.

Las estadísticas aceptan `?desde=` y `?hasta=` (`AAAA-MM-DD`, ambos incluidos; sin rango, los últimos 14 días) y devuelven las series completas, con 0 en los días sin datos.

7. Seguridad

La API no confía en nada de lo que llega: valida cada dato, comprueba el rol en cada pedido y limita lo que una misma conexión puede hacer. El 27/10 se revisó con pruebas de ataque contra todos los endpoints: se corrigieron 16 problemas y hoy esas pruebas forman parte de los tests. El informe completo está en `SECURITY.md` del repositorio.

Área	Qué se hace
Contraseñas	Se guardan con hash (bcrypt). Mínimo de 8 caracteres con mayúscula, minúscula y número.
Sesión	Token firmado (JWT, HS256) de 7 días. Cambiar o recuperar la contraseña cierra las demás sesiones. El enlace de recuperación es de un solo uso, vence en 1 hora y en la base solo queda su hash.
Login con Google	Se verifica el token con Google y solo se acepta un email verificado. Si el email ya tenía una cuenta con contraseña, esa contraseña se conserva: las dos formas de entrar conviven. Como el registro no verifica el email, al vincular Google se cierran las sesiones que ya estuvieran abiertas (riesgo anotado en SECURITY.md).
Fuerza bruta	5 intentos fallidos por cuenta y conexión bloquean 15 minutos (quien falla desde otro lugar no deja afuera al dueño); 50 en total bloquean la cuenta para todos; 20 fallos por conexión bloquean esa conexión. El intento se cuenta antes de revisar la clave, así que los pedidos en paralelo no dan más intentos.
Límites de pedidos	300 por minuto por conexión en toda la API; 20 registros por hora; 5 consultas por hora y 3 por hora para un mismo email destinatario; 3 adjuntos por hora; recuperación de contraseña limitada por conexión y por email. Pasados, responde 429.

Área	Qué se hace
Autorización	El rol se lee de la base en cada pedido. Las 28 rutas protegidas se probaron con visitante, usuario, operador y admin. Un usuario solo ve sus consultas y un borrador es un 404 para quien no es admin.
Validación	Cada endpoint valida con las reglas del contrato y quita las etiquetas HTML de los textos. Solo se aceptan textos donde se espera un texto (así no entran operadores de MongoDB), los campos desconocidos se ignoran y el cuerpo no puede pasar de 100 KB.
Archivos	El tipo se decide por el contenido real del archivo, no por la extensión: solo JPG, PNG y WebP (y PDF en los adjuntos), hasta 2 MB. Un adjunto de una consulta tiene que ser un archivo subido a nuestra cuenta de Cloudinary.
Emails	Todo texto que escribe una persona se escapa en el HTML, los asuntos no pueden inyectar encabezados y los enlaces usan la URL configurada del cliente. En producción sin servidor de emails no se simula el envío: falla, para no dejar enlaces de recuperación en el registro.
Red y encabezados	`helmet` (HSTS y demás), CORS con una lista de orígenes permitidos, `Cache-Control: no-store` en toda la API y errores genéricos sin detalles técnicos.
Dependencias y secretos	`npm audit` sin vulnerabilidades. El `.env` no se sube al repositorio, las variables se validan al arrancar y en producción la clave de firma tiene que tener al menos 32 caracteres.

Riesgos que quedan

- El token de sesión vive en `localStorage` del navegador (así lo define el plan): una falla de XSS en el cliente lo expondría. El cliente debe evitar `dangerouslySetInnerHTML` y publicarse con un `Content-Security-Policy`.
- El registro no verifica el email con un mensaje de confirmación.
- Los límites de pedidos viven en la memoria del servidor y se reinician con cada deploy.
- No hay lista de contraseñas comunes ni segundo factor.

- Las cuentas de prueba (`admin@portfolio.test`, `operador@portfolio.test` y `usuario@portfolio.test`) tienen email conocido: hay que cambiar su contraseña cuando termine la evaluación.

8. Despliegue y configuración

Cada pieza se publica sola con un push a `main`: Vercel construye el cliente y Render construye y arranca la API. Todo es configuración por variables de entorno; no hay claves en el código. El paso a paso está en `DEPLOY.md` del repositorio y todavía no se ejecutó.

La API en Render

Campo	Valor
Carpeta raíz	`server`
Instalación	`npm ci --omit=dev` (sin las herramientas de test, que bajan un binario de MongoDB)
Arranque	`npm start`
Comprobación de salud	`/api/health`
Plan	Gratuito

Al arrancar, el servidor valida su configuración y se niega a iniciar si falta algo o está mal: las variables obligatorias, una clave de firma de al menos 32 caracteres en producción o una `CLIENT_URL` que no sea una URL. Si MongoDB no responde en 10 segundos, falla con un mensaje claro, y cierra ordenadamente cuando Render lo reinicia.

Variables de entorno del servidor

Variable	Para qué sirve
`MONGODB_URI`	Conexión a MongoDB Atlas. Obligatoria.
`JWT_SECRET`	Clave con la que se firman las sesiones. Obligatoria.
`CLIENT_URL`	URL del cliente: define el CORS y los enlaces de los emails. Admite varias, con comodines para las previsualizaciones de Vercel.
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USER`, `MAIL_PASS` y `MAIL_FROM`	Servidor de emails (SMTP). Sin `MAIL_HOST` se usa Gmail.

Variable	Para qué sirve
`CLOUDINARY_URL`	Subida de archivos. Sin esto la subida responde 503.
`GOOGLE_CLIENT_ID`	Login con Google. Sin esto responde 503.
`TRUST_PROXY`	Cuántos proxies hay delante del servidor, para que los límites cuenten por visitante.
`NODE_ENV`	`production` en Render: activa las validaciones más estrictas.
`SEED_PASSWORD`	Contraseña de las cuentas de prueba. Solo se usa en tu PC al correr el seed.

El cliente en Vercel

- Variables: `VITE_API_URL` (la URL de la API en Render) y `VITE_GOOGLE_CLIENT_ID`.
- Un `vercel.json` que reescribe todas las rutas a `index.html`; sin él, abrir directamente un enlace como `/recuperar?token=…` da 404.
- En Google Cloud, la URL de Vercel va en los orígenes autorizados del Client ID.

Datos de prueba y verificación

- `npm run seed` carga datos que reproducen el prototipo: 40 usuarios, 6 proyectos, 4 servicios, 30 consultas y 30 días de visitas y accesos, para que el dashboard se vea completo. Contra una base remota exige una contraseña propia y la confirmación `--yes`, porque borra todo.
- Las cuentas de prueba son una por rol: `admin@portfolio.test`, `operador@portfolio.test` y `usuario@portfolio.test`. Su contraseña se entrega a la profesora por fuera del repositorio.
- `npm run smoke` prueba una API publicada (salud, CORS, datos, rutas protegidas, login y dashboard) y marca cada comprobación.

Limitaciones del plan gratuito

- Render duerme el servidor tras 15 minutos sin tráfico y tarda cerca de 1 minuto en despertar: hay que abrirlo unos minutos antes de la demo.
- Render bloquea los puertos de salida 25, 465 y 587, que son los de Gmail. Por eso en producción los emails salen por un servicio SMTP con el puerto 2525.
- 750 horas gratis por mes alcanzan para un solo servicio prendido todo el mes.

9. Pruebas

El servidor tiene 246 tests automáticos que corren en unos 30 segundos con `npm test`, sin internet ni una base instalada: usan una MongoDB en memoria, un Cloudinary falso y un Google falso. Cada test hace pedidos HTTP reales contra la aplicación.

Archivo	Tests	Qué cubre
`auth.test.js`	31	Registro, login y bloqueo, cuenta propia, recuperar contraseña, roles y sesión opcional.
`content.test.js`	27	Perfil, proyectos y servicios: lecturas públicas, CRUD de admin y listados de admin.
`inquiries.test.js`	30	Enviar consultas, bandeja con filtros, detalle, estado, responder por email y «mis consultas».
`uploads.test.js`	15	Subida de archivos, tipos y pesos, y borrado de imágenes que dejan de usarse.
`users.test.js`	14	Gestión de usuarios y sus protecciones.
`visits.test.js`	7	Conteo de visitas, incluso con pedidos simultáneos.
`stats.test.js`	15	Estadísticas del dashboard, con los bordes de hora de Salta.
`seed.test.js`	12	Que los datos de prueba tengan las cantidades pedidas y que el dashboard se vea con vida.
`deploy.test.js`	21	Validación de la configuración, CORS para Vercel, configuración del email y protecciones del seed.
`security.test.js`	66	Ataques: acceso por rol en 28 rutas, tokens falsos, inyección de operadores de MongoDB, asignación masiva, fuga de datos, archivos peligrosos, fuerza bruta y texto malicioso.
`security-limits.test.js`	8	Límites de pedidos por conexión y por destinatario.

Además de los tests, el servidor se probó corriendo en modo producción con los datos de prueba: login, dashboard, CORS, bloqueo de un atacante sin afectar al dueño y el script `npm run smoke`.

Lo que no se pudo probar fuera del despliegue real: MongoDB Atlas, el login con Google de verdad, Cloudinary y el envío de emails con credenciales reales, y cuántos proxies tiene Render delante del servidor.

10. Decisiones de diseño y límites conocidos

Estas son las decisiones que dan forma al sistema, con la razón de cada una.

Decisión	Por qué
API REST propia en Express, con el cliente separado	Es lo que piden la consigna y las clases 1 y 2, y deja que el cliente y el servidor se desplieguen y evolucionen por separado.
Base documental (MongoDB) con las respuestas anidadas en cada consulta	Las respuestas se leen siempre junto a su consulta (clase 9: objetos anidados) y no hacen falta joins.
El rol se lee de la base en cada pedido, no del token	Un cambio de rol o una cuenta desactivada valen al instante, a costa de una lectura por pedido.
Sesión con JWT en el encabezado `Authorization`	El cliente (Vercel) y la API (Render) están en dominios distintos; con cookies habría que resolver CORS con credenciales y CSRF. El costo es que el token queda en `localStorage` (ver la sección 7).
Validación propia, con los mensajes del prototipo	Cada campo devuelve su mensaje en voseo, igual al del prototipo, y se valida siempre en el servidor aunque el cliente ya lo haya hecho.
Archivos en Cloudinary y no en disco	El disco de Render se borra en cada reinicio.
Emails por un servicio SMTP configurable	Render gratis bloquea los puertos de Gmail; el mismo código sirve con Gmail en desarrollo.
Visitas por sesión y página, con un índice único	Recargar no infla el contador, ni siquiera con pedidos simultáneos.
Los días de las estadísticas se cuentan en hora de Salta	Los gráficos coinciden con lo que ve el administrador, no con UTC.
`bcryptjs` en lugar de `bcrypt`	No necesita compilar código nativo en Windows ni en Render.
Un solo repositorio con `client/` y `server/`	Un solo lugar para el código, las revisiones y los despliegues automáticos.

Pendiente y fuera de alcance

- Publicar la API en Render y el cliente en Vercel, y ejecutar `npm run smoke` contra lo publicado.
- Cerrar qué framework CSS se usa en el cliente: el plan dice Tailwind y el manual dice Bootstrap 5.
- Agregar el `vercel.json` del cliente para que funcionen los enlaces directos.
- Definir la contraseña de las cuentas de prueba y entregarla a la profesora.
- Verificación del email al registrarse, segundo factor y almacenamiento compartido de los límites de pedidos quedan fuera de este trabajo.

11. Equipo y reparto de áreas

Trabajo del Grupo 3 de Lenguajes IV, Universidad Católica de Salta, Facultad de Ingeniería, 2026. Docente: Ing. Nancy Lucero. Cada integrante es dueño de un área y todos programan.

Área	Integrante	Responsabilidad
A. Diseño, UX y animaciones	María Gimena Cruz	Manual de marca, prototipo navegable, animaciones, diseño responsive y revisión visual.
B. Frontend público	Bernardo Pérez del Cerro	Sitio del visitante y del usuario registrado, formulario de contacto, proyectos, login y registro.
C. Backend, base de datos y autenticación	Ezequiel Austerlitz Palavecino	API REST, MongoDB, sesiones y roles, emails, archivos, estadísticas, despliegue del servidor y este documento.
D. Paneles de operador y administrador	Francisco Rafael Vega	Bandeja y respuestas del operador, CRUD de proyectos y servicios, usuarios y dashboard con gráficos.